Why we almost needed an NP-hard algorithm (and why that's not a red flag)

A teaching note, not just a finding: the goal here is to understand *why* adding "more than one client" can tip a textbook-easy problem into NP-hard territory — precisely, not just by gesturing at "it gets harder."

1. Where we started: a problem Dijkstra solves perfectly

One client, one request: *find the cheapest path from this client to the server that should serve it.* This is the classical single-source shortest-path problem. Dijkstra's algorithm solves it exactly, in polynomial time ($\Theta(V^2)$ the way it's implemented in `algorithms/dijkstra.py`, or $O((V+E) \log V)$ with a binary heap), and there is nothing to debate about whether it's "good enough" — it provably finds the optimum, every time. This is the entire scope of `PathFinder` as built in this package, and it is the right tool for exactly this problem.

2. What changes the moment there's a second client

Route a second client at the same time, and the problem is no longer "find a cheap path" — it's "find a cheap path **that doesn't fight the first client for the same link**." Concretely: every link has a finite capacity (bandwidth); if client A's path and client B's path share a link, their combined demand on that link must not exceed what it can carry. This is the **multi-commodity flow problem**: given several (source, target, demand) triples ("commodities") on one graph with capacities, assign each a path so that no link's capacity is exceeded, minimizing total cost.

This single addition — a resource shared *across* otherwise-independent subproblems — is exactly what turns "two easy problems" into "one hard problem." It's a pattern that shows up constantly in CS: scheduling one job on one machine is trivial; scheduling many jobs on machines that share a resource is NP-hard (job-shop scheduling). Packing one item is nothing; packing many items into bins of fixed size is NP-hard (bin packing). Here: routing one flow is nothing; routing many flows over links of fixed capacity is the same story.

3. The fork in the road: can a client's traffic be split?

This is where the complexity class is actually decided, and it's worth being precise, because the two cases are not "equally hard, just one has a bigger solver":

- **If a client's traffic may be split across several paths** (some percentage this way, the rest that way), the problem is the *fractional* multi-commodity flow problem: flow-conservation and capacity constraints are all linear, the objective is linear, and the variables are continuous (a flow *amount*, not a 0/1 choice). This is a **linear program**, and linear programs are solvable in polynomial time — a landmark, well-established result (ellipsoid method, Khachiyan 1979; practical interior-point and simplex-family methods used ubiquitously since). Splittable multi-commodity flow is, complexity-wise, an *easy* problem.
- **If each client's traffic must travel as one undivided path** (no splitting — which is what the issue this package is responding to actually asks for: "an ordered list of nodes or links," one path, not a fractional mix), the variables become integer (0/1: is this edge on this client's path or not), and the feasible region is no longer a nice convex shape an LP solver can walk across — it's a combinatorial choice of one path per client, among exponentially many, under a constraint that couples every client's choice to every other's. This is **integer** linear programming, and this specific case is a named, studied result: **Even, Itai, and Shamir (1976)** proved that the integral two-commodity flow problem is NP-complete — in both the directed and undirected case — with as few as **two** commodities.[1] Not "many clients eventually makes it hard" — *two* is already enough, as soon as paths can't be split.

So the honest, precise answer to "why does this need ILP" is: **it's the no-splitting requirement, combined with a shared capacity constraint, that creates the hardness — not simply "more than one client."** Splittable multi-commodity flow with a thousand clients is still polynomial. Unsplittable multi-commodity flow with two clients is already NP-complete.

4. Why Dijkstra specifically cannot be patched for this

It's worth being precise about *why* Dijkstra breaks here, rather than just "it doesn't know about capacity." Dijkstra's correctness proof rests on one structural guarantee: once a node is popped with its final distance, that distance can never improve later, *because every edge weight is a fixed number, known in advance, that does not depend on any other decision being made.* That's the invariant that makes the greedy choice safe.

The moment a second client competes for the same link, an edge's usability for client A now depends on whether client B's (not-yet-decided) path also wants it. The "cost," or even the availability, of an edge is no longer a static input to the algorithm — it's an output of the very decision the algorithm is supposed to help make, for a different commodity entirely. There is no way to bolt that onto Dijkstra's loop; its entire mechanism assumes the quantity it's optimizing over (edge weight) doesn't move while it's running. This is also why running Dijkstra once per client, independently, doesn't fix anything (the "sequential trap"): each call still privately assumes its edge weights are fixed and final, when in reality the *previous* client's routing choice has silently used up capacity the *next* call has no way of seeing.

5. Why reach for ILP anyway, if it's NP-hard?

Because "NP-hard" describes the *worst case*, not every instance. Modern ILP solvers (OR-Tools/SCIP, CPLEX, Gurobi) don't try to beat the worst-case bound — they use branch-and-bound: solve the easy LP relaxation for a bound, then intelligently search only the branches that could beat it, pruning the rest. On real, modestly-sized instances (here: a handful of concurrent clients, a few dozen PoPs) this routinely finds the exact optimum in milliseconds, which is why `cdn_qoe.py` reaching for OR-Tools isn't engineering overkill *for the multi-client version of this problem* — it's the standard, correct tool once the no-splitting + shared-capacity combination is actually in play. (It's only overkill, per `FINDINGS.md`, for the version of the problem the code **currently** solves — one client, one choice among candidate servers, no capacity constraint at all — which needs none of this machinery.)

6. The broader lesson

A problem doesn't have to look complicated to become NP-hard — it just has to stop being decomposable into independent pieces. One shortest path is easy because nothing else in the system depends on the choice made. Introduce a shared, finite resource that several such choices must now divide between them, and the problem can jump complexity classes with no other change at all — not because the individual sub-problems got harder, but because they stopped being separable. That's the actual reason "Dijkstra once per client" is unsound and "Dijkstra joint over all clients" doesn't exist: the problem that would need solving is a different, harder problem, not a bigger version of the same one.

7. Where this leaves the actual implementation

Nothing here changes what's already built. `PathFinder`/`DijkstraPathFinder` remain exactly correct for the problem REIN's deployer solves *today* — one client, routed independently, no shared-capacity coordination attempted. The multi-client, capacity-aware version is real future work, belongs behind its own interface (not a fourth `PathFinder`, per `FINDINGS.md`'s LSP argument), and when it's built, reaching for an ILP solver for it will be the mathematically correct call, not a heavier-than-necessary one.

---

1. S. Even, A. Itai, A. Shamir, "On the Complexity of Timetable and Multicommodity Flow Problems," *SIAM Journal on Computing*, 5(4):691–703, 1976.